

PLANO COMPLETO MVP - SISTEMA DE LAUDOS EEG COM IA

6 Meses - Do Zero ao Produto Funcional

ÍNDICE

1. Visão Geral do Projeto
 2. Arquitetura Técnica
 3. Cronograma Detalhado
 4. Equipe e Responsabilidades
 5. Tecnologias
 6. Metodologia de Trabalho
 7. Entregáveis por Mês
 8. Riscos e Mitigações
 9. Orçamento
 10. Critérios de Sucesso
-

1 VISÃO GERAL DO PROJETO

Objetivo

Desenvolver um sistema web (PWA) que utiliza Inteligência Artificial para auxiliar médicos na elaboração de laudos de eletroencefalograma (EEG), reduzindo o tempo de análise de 20-30 minutos para 10 minutos por exame.

Proposta de Valor

Para o Médico:

-  **Redução de 50-60% no tempo** de elaboração de laudos
-  **Detecção assistida** de padrões anormais (IA aponta, médico valida)
-  **Laudos pré-estruturados** em linguagem médica profissional
-  **Redução de fadiga** mental ao final do dia
-  **Consistência** nos laudos (padrão de qualidade)

Para a Clínica/Hospital:

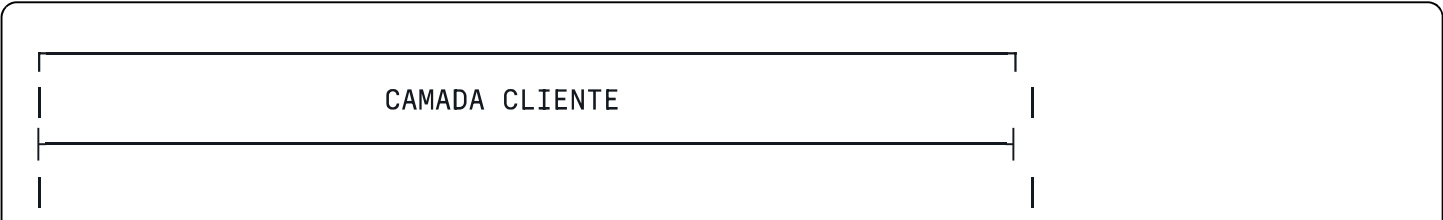
-  **Aumento de produtividade** em 2-3x por médico
-  **Mais laudos = mais faturamento**
-  **Diferencial competitivo** no mercado
-  **Padronização** de qualidade
-  **Satisfação** profissional da equipe médica

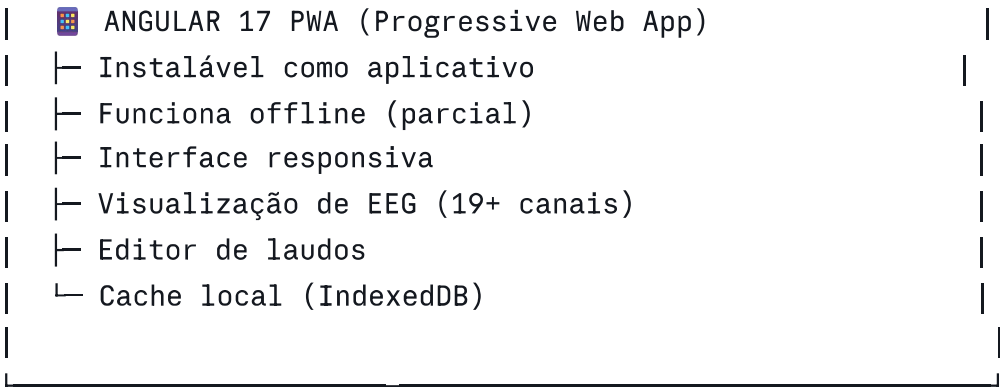
 Diferencial Competitivo

1. **IA Aprende do Sinal, Não Copia Laudos**
 - Sistema analisa o exame diretamente
 - Desenvolve capacidade própria de detecção
 - Melhora continuamente com uso
2. **Funcionalidade Offline (Híbrido)**
 - PWA instalável como aplicativo
 - Trabalha com dados em cache quando sem internet
 - Sincroniza automaticamente quando volta online
3. **Linguagem Médica Brasileira**
 - Terminologia técnica correta em português
 - Estrutura de laudo padrão brasileiro
 - Adequado à prática médica local
4. **Melhoria Contínua Automática**
 - Cada correção do médico treina a IA
 - Sistema fica mais preciso com o tempo
 - Aprende com múltiplos médicos

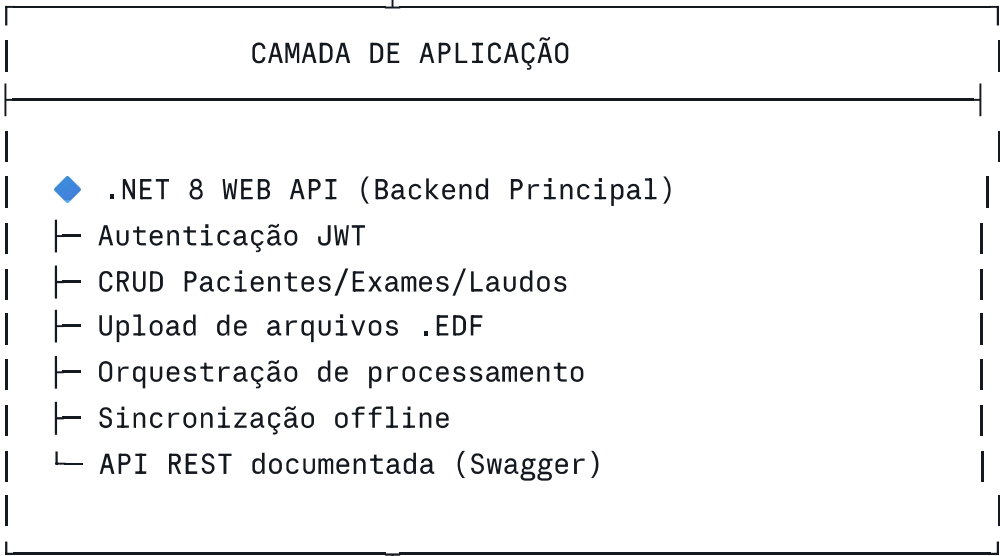
 ARQUITETURA TÉCNICA

 Diagrama de Arquitetura

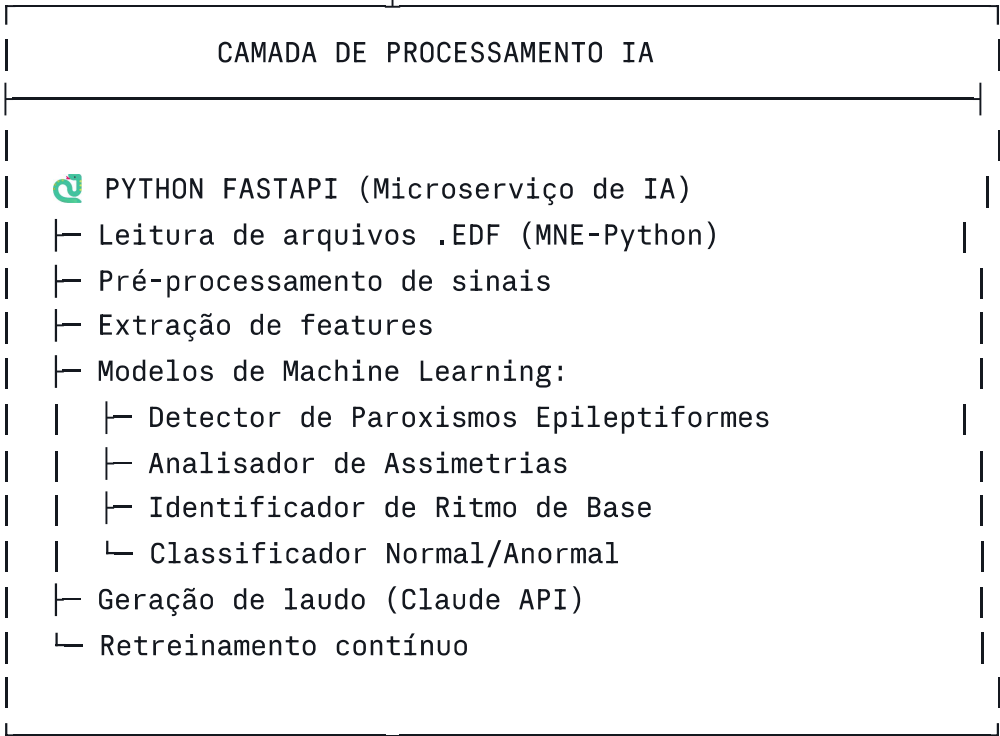




HTTPS / REST



Chamadas HTTP/gRPC



CAMADA DE DADOS

-  PostgreSQL 15
 - └ Pacientes
 - └ Exames
 - └ Laudos
 - └ Marcações (training data)
 - └ Usuários
 - └ Logs de auditoria
-  Azure Blob Storage / AWS S3
 - └ Arquivos .EDF originais
 - └ PDFs de laudos finalizados
 - └ Modelos de IA treinados
-  Redis (Opcional - Fase 2)
 - └ Cache de sessões

Segurança

Autenticação e Autorização:

- JWT (JSON Web Tokens) para autenticação
- Refresh tokens para sessões longas
- RBAC (Role-Based Access Control):
 - Admin (gestão completa)
 - Médico (laudos e análises)
 - Técnico (upload de exames)

Proteção de Dados:

- HTTPS obrigatório (TLS 1.3)
- Criptografia de dados sensíveis em repouso (AES-256)
- Compliance com LGPD:
 - Consentimento explícito
 - Direito ao esquecimento
 - Logs de acesso
 - Anonimização de dados de treino

Auditoria:

- Log de todas ações críticas
- Rastreabilidade completa (quem fez o quê, quando)
- Backup automático diário
- Retenção de dados conforme legislação médica

3 CRONOGRAMA DETALHADO (6 MESES)

📅 MÊS 1: FUNDAÇÃO E INFRAESTRUTURA

Semana 1-2: Setup de Projeto

Responsável: Tech Lead + Equipe completa

Atividades:

Backend (.NET):

- ☐ Criar projeto .NET 8 Web API
- ☐ Estruturar em Clean Architecture:

EEG.Domain/

Entidades e interfaces

EEG.Application/

Casos de uso

EEG.Infrastructure/

Implementações

EEG.API/

Controllers

- ☐ Setup Entity Framework Core + PostgreSQL
- ☐ Configurar Swagger/OpenAPI
- ☐ Implementar autenticação JWT básica
- ☐ Setup de testes unitários (xUnit)

Frontend (Angular):

- ☐ Criar projeto Angular 17
- ☐ Setup PWA (@angular/pwa)
- ☐ Estrutura de módulos:

auth/	# Autenticação
patients/	# Gestão de pacientes
exams/	# Exames
reports/	# Laudos
shared/	# Componentes compartilhados

- ☐ Configurar Angular Material
- ☐ Setup de testes (Jasmine/Karma)

IA (Python):

- ☐ Criar projeto FastAPI
- ☐ Setup ambiente Python (Poetry/venv)
- ☐ Instalar dependências:

```
mne-python==1.6.0
numpy==1.25.0
scipy==1.11.0
scikit-learn==1.3.0
tensorflow==2.14.0 # ou PyTorch
fastapi==0.104.0
```

- ☐ Setup de testes (pytest)

Infraestrutura:

- ☐ Configurar repositório Git (GitHub/GitLab)
- ☐ Setup CI/CD básico (GitHub Actions)
- ☐ Criar ambientes: Dev, Staging, Prod
- ☐ Configurar Docker/Docker Compose local

Entregável Semana 2: ☒ Projeto estruturado e rodando localmente ☒ Pipeline CI/CD básico funcionando ☒ Documentação inicial criada

Semana 3-4: Importação de Arquivos EEG

Responsável: Dev Backend + Dev Python

Objetivo: Sistema consegue ler arquivos .EDF reais

Atividades:

Backend (.NET):

- ☐ Controller de upload de arquivos

- ☐ Validação de formato .EDF
- ☐ Armazenamento em Blob Storage
- ☐ Criação de registro no banco (Exam entity)
- ☐ API endpoint: `POST /api/exams/upload`

Python:

- ☐ Endpoint FastAPI: `POST /api/process/validate-edf`
- ☐ Leitura de .EDF com MNE-Python:

```
python

import mne

def read_eeg_file(file_path):
    raw = mne.io.read_raw_edf(file_path, preload=True)

    metadata = {
        'n_channels': raw.info['nchan'],
        'sampling_rate': raw.info['sfreq'],
        'duration': raw.times[-1],
        'channel_names': raw.ch_names
    }

    return raw, metadata
```

- ☐ Extração de metadados:
 - Nome do paciente
 - Data do exame
 - Duração
 - Número de canais
 - Taxa de amostragem
- ☐ Validação de integridade do arquivo

Frontend (Angular):

- ☐ Tela de upload de exame
- ☐ Componente de drag-and-drop
- ☐ Barra de progresso de upload
- ☐ Validação client-side (tamanho, formato)
- ☐ Exibição de metadados após upload

Testes:

- ☐ Testar com arquivo real do Isaac (fornecido pelo médico)

- ☐ Testar com arquivo corrompido (deve rejeitar)
- ☐ Testar com formato errado (deve rejeitar)

Entregável Semana 4: ☒ Upload de .EDF funciona end-to-end ☒ Metadados extraídos corretamente ☒ Arquivo armazenado com segurança

Semana 5-8: Visualização de EEG

Responsável: Dev Frontend + Dev Python

Objetivo: Exibir os 19 canais do EEG navegáveis

Atividades:

Python:

- ☐ Endpoint: `GET /api/process/eeg-data/{exam_id}`
- ☐ Retornar dados do sinal em formato JSON:

python

```
def get_eeg_segment(exam_id, start_time, duration):
    raw = load_eeg(exam_id)

    # Extrai segmento
    start_sample = int(start_time * raw.info['sfreq'])
    end_sample = int((start_time + duration) * raw.info['sfreq'])
    data = raw.get_data(start=start_sample, stop=end_sample)

    return {
        'channels': raw.ch_names,
        'data': data.tolist(),
        'timestamps': raw.times[start_sample:end_sample].tolist(),
        'sampling_rate': raw.info['sfreq']
    }
```

- ☐ Paginação de dados (retornar janelas de 10 segundos)
- ☐ Pré-processamento básico aplicado

Frontend (Angular):

- ☐ Componente de visualização EEG:

typescript

```

@Component({
  selector: 'app-eeg-viewer',
  template: `
    <div class="eeg-container">
      <div class="controls">
        <button (click)="previous()"><◀ Anterior</button>
        <span>{{ currentTime }}s / {{ totalDuration }}s</span>
        <button (click)="next()">Próximo ▶</button>
        <input type="range"
          [(ngModel)]="currentTime"
          (change)="loadSegment()"
          [max]="totalDuration">
      </div>

      <canvas #eegCanvas></canvas>
    </div>
  `
})
export class EegViewerComponent {
  @ViewChild('eegCanvas') canvas: ElementRef;

  // Lógica de renderização
}

```

☐ Renderização usando Canvas (performance)

☐ Controles de navegação:

- Avançar/retroceder
- Timeline slider
- Zoom in/out

☐ Ajuste de escala de amplitude

☐ Exibição de 19 canais simultâneos

☐ Labels dos canais (Fp1-F3, F3-C3, etc)

Bibliotecas de Gráficos (escolher uma):

- Opção A: Plotly.js (mais fácil, menos performance)
- Opção B: Canvas nativo (mais performance, mais trabalho)
- Opção C: Chart.js (meio termo)

Recomendação: Canvas nativo para melhor performance

Testes:

☐ Renderizar exame do Isaac completo (60 min)

- ☐ Navegar por todo o exame sem travar
- ☐ Verificar performance (renderizar 10s em < 1s)
- ☐ Testar zoom e ajuste de escala

Entregável Semana 8: ☒ Visualização completa dos 19 canais ☒ Navegação fluida por todo o exame ☒ Performance adequada (sem travamentos) ☒ Interface intuitiva

MÊS 2: PRÉ-PROCESSAMENTO E MARCAÇÕES

Semana 9-10: Pré-processamento de Sinais

Responsável: Dev Python + Cientista de Dados

Objetivo: Limpar e preparar sinal para análise

Atividades:

Filtros:

- ☐ Implementar filtro passa-banda (0.5-70 Hz):

python

```
from scipy import signal

def apply_bandpass_filter(raw_data, sfreq, lowcut=0.5, highcut=70):
    nyquist = sfreq / 2
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = signal.butter(4, [low, high], btype='band')
    filtered = signal.filtfilt(b, a, raw_data)
    return filtered
```

- ☐ Implementar filtro notch (60 Hz - rede elétrica):

python

```
def apply_notch_filter(raw_data, sfreq, freq=60):
    b, a = signal.iirnotch(freq, Q=30, fs=sfreq)
    filtered = signal.filtfilt(b, a, raw_data)
    return filtered
```

Re-referenciamento:

- ☐ Implementar montagem bipolar (banana)
- ☐ Implementar montagem de referência comum

☐ Permitir seleção de montagem

Detecção de Artefatos:

- ☐ Detector de artefatos de movimento (amplitude > threshold)
- ☐ Detector de artefatos de piscar (canais frontais)
- ☐ Detector de artefatos musculares (alta frequência)
- ☐ Marcar trechos com artefatos (não remover)

Segmentação:

- ☐ Dividir sinal em épocas de 1-2 segundos
- ☐ Criar janelas deslizantes (overlap de 50%)

Testes:

- ☐ Comparar sinal original vs filtrado visualmente
- ☐ Validar que artefatos óbvios são detectados
- ☐ Verificar que filtros não distorcem padrões importantes

Entregável Semana 10: ☒ Pipeline de pré-processamento completo ☒ Sinal limpo e pronto para análise ☒ Artefatos identificados (mas não removidos)

Semana 11-12: Interface de Marcação para Médicos

Responsável: Dev Frontend + Dev Backend

Objetivo: Permitir que médico marque padrões nos exames

Atividades:

Frontend (Angular):

- ☐ Modo de marcação na visualização EEG:

typescript

```
// Componente de marcação
@Component({
  selector: 'app-marking-tool',
  template: `
    <div class="marking-mode">
      <h3>Marcar Padrão Anormal</h3>

      <!-- Seletor de tipo -->
      <mat-button-toggle-group [(ngModel)]="selectedType">
        <mat-button-toggle value="spike">Ponta</mat-button-toggle>
        <mat-button-toggle value="sharp_wave">Onda Aguda</mat-button-toggle>
        <mat-button-toggle value="slow_wave">Onda Lenta</mat-button-toggle>
        <mat-button-toggle value="artifact">Artefato</mat-button-toggle>
      </mat-button-toggle-group>

      <!-- Severidade -->
      <mat-radio-group [(ngModel)]="severity">
        <mat-radio-button value="mild">Leve</mat-radio-button>
        <mat-radio-button value="moderate">Moderada</mat-radio-button>
        <mat-radio-button value="severe">Severa</mat-radio-button>
      </mat-radio-group>

      <!-- Notas -->
      <mat-form-field>
        <textarea matInput
          placeholder="Notas (opcional)"
          [(ngModel)]="notes"></textarea>
      </mat-form-field>

      <button (click)="saveMarking()">Salvar Marcação</button>
    </div>
  `
})
```

- ☐ Click no gráfico para marcar evento
- ☐ Seleção de intervalo (arrastar mouse)
- ☐ Lista de marcações criadas
- ☐ Edição/remoção de marcações
- ☐ Marcação de assimetrias globais
- ☐ Marcação de ritmo de base

Backend (.NET):

- ☐ Entity: Marking

csharp

```
public class Marking
{
    public Guid Id { get; set; }
    public Guid ExamId { get; set; }
    public double TimestampSeconds { get; set; }
    public double DurationSeconds { get; set; }
    public string[] Channels { get; set; }
    public MarkingType Type { get; set; } // Spike, SharpWave, etc
    public Severity Severity { get; set; }
    public string MarkedBy { get; set; }
    public string Notes { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

☐ CRUD de marcações:

- POST /api/markings
- GET /api/markings/{examId}
- PUT /api/markings/{id}
- DELETE /api/markings/{id}

Testes:

- ☐ Médico marca 5 eventos no exame do Isaac
- ☐ Marcações são salvas corretamente
- ☐ Marcações aparecem no gráfico
- ☐ Edição e remoção funcionam

Entregável Semana 12: ☒ Interface de marcação funcional ☒ Médico consegue marcar padrões facilmente ☒ Dados salvos no formato correto para treino IA



MÊS 3-4: DESENVOLVIMENTO DOS MODELOS DE IA

Semana 13-16: Modelo 1 - Detector de Paroxismos Epileptiformes

Responsável: Cientista de Dados + Dev Python

Objetivo: IA detecta pontas e ondas agudas

Dataset Necessário:

- ☐ Coletar 200-500 eventos marcados pelo médico:
 - 150 Pontas (spikes)

- 150 Ondas agudas (sharp waves)
- 200 Trechos normais (controle negativo)

Atividades:

Extração de Features:

python

```
def extract_spike_features(signal_window, sfreq):  
    """  
    Extrai features de uma janela de 200ms  
    """  
    features = {}  
  
    # Domínio do tempo  
    features['peak_amplitude'] = np.max(np.abs(signal_window))  
    features['duration'] = len(signal_window) / sfreq  
    features['slope_up'] = calculate_slope(signal_window, 'rising')  
    features['slope_down'] = calculate_slope(signal_window, 'falling')  
    features['asymmetry'] = calculate_asymmetry(signal_window)  
  
    # Domínio da frequência  
    freqs, psd = signal.welch(signal_window, sfreq)  
    features['dominant_freq'] = freqs[np.argmax(psd)]  
    features['spectral_centroid'] = np.sum(freqs * psd) / np.sum(psd)  
  
    # Morfologia  
    features['sharpness'] = calculate_sharpness(signal_window)  
    features['spike_score'] = classify_morphology(signal_window)  
  
    return features
```

Modelo (escolher abordagem):

Opção A: Machine Learning Clássico (mais rápido de implementar)

python

```
from sklearn.ensemble import RandomForestClassifier  
  
# Treinar  
X_train = [extract_spike_features(window) for window in training_windows]  
y_train = [marking.type for marking in markings]  
  
model = RandomForestClassifier(n_estimators=100)  
model.fit(X_train, y_train)
```

Opção B: Deep Learning (melhor performance)

```
python

import tensorflow as tf

def build_spike_detector():
    model = tf.keras.Sequential([
        tf.keras.layers.Conv1D(32, 5, activation='relu', input_shape=(200, 19)),
        tf.keras.layers.MaxPooling1D(2),
        tf.keras.layers.Conv1D(64, 5, activation='relu'),
        tf.keras.layers.MaxPooling1D(2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(3, activation='softmax') # Normal, Spike, Sharp Wave
    ])

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model
```

Recomendação Fase 1: Random Forest (mais rápido, bom baseline)

Treinamento:

- ☐ Split dataset: 70% treino, 15% validação, 15% teste
- ☐ Cross-validation (5-fold)
- ☐ Otimização de hiperparâmetros
- ☐ Avaliar métricas:
 - Sensibilidade (recall): > 80%
 - Especificidade: > 70%
 - Precisão: > 75%

Integração:

- ☐ Endpoint FastAPI: `POST /api/detect/spikes`
- ☐ Processar exame completo
- ☐ Retornar lista de eventos detectados

Testes:

- ☐ Testar com exame do Isaac (deve encontrar os paroxismos em F7-T3)
- ☐ Calcular confusion matrix
- ☐ Validar com médico

Entregável Semana 16: ☒ Modelo treinado com acurácia > 75% ☒ API de detecção funcionando ☒ Testado em casos reais

Semana 17-18: Modelo 2 - Analisador de Assimetrias

Responsável: Cientista de Dados + Dev Python

Objetivo: Detectar diferenças entre hemisférios

Dataset Necessário:

- ☐ 50 exames com assimetrias marcadas
- ☐ 50 exames simétricos normais

Abordagem:

```
python
```

```
def analyze_asymmetry(eeg_data, channel_names):
    """
    Compara hemisférios esquerdo vs direito
    """
    # Canais por hemisfério
    left_channels = ['F3', 'C3', 'P3', 'O1', 'F7', 'T3', 'T5']
    right_channels = ['F4', 'C4', 'P4', 'O2', 'F8', 'T4', 'T6']

    # Extrair dados
    left_data = extract_channels(eeg_data, left_channels)
    right_data = extract_channels(eeg_data, right_channels)

    # Calcular potência espectral por banda
    left_bands = calculate_band_power(left_data, sfreq)
    right_bands = calculate_band_power(right_data, sfreq)

    # Comparar
    asymmetry_score = {}
    for band in ['delta', 'theta', 'alpha', 'beta']:
        diff = abs(left_bands[band] - right_bands[band])
        avg = (left_bands[band] + right_bands[band]) / 2
        asymmetry_score[band] = diff / avg # % de diferença

    # Decisão
    is_asymmetric = any(score > 0.3 for score in asymmetry_score.values())
    predominant_side = 'left' if left_bands['power_total'] > right_bands['power_total']

    return {
        'asymmetric': is_asymmetric,
        'predominant_side': predominant_side,
        'scores': asymmetry_score
    }
```

Entregável Semana 18: ☒ Detector de assimetrias funcionando ☒ Identifica corretamente hemisfério predominante ☒ Testado no caso Isaac (deve detectar esquerdo)

Semana 19-20: Modelo 3 - Identificador de Ritmo de Base

Responsável: Cientista de Dados + Dev Python

Objetivo: Identificar frequência do ritmo alfa posterior

Abordagem:

python

```
def identify_base_rhythm(eeg_data, channel_names, age_years):
    """
    Identifica ritmo de base nas regiões posteriores
    """
    # Canais posteriores
    posterior_channels = ['01', '02', 'P3', 'P4']
    posterior_data = extract_channels(eeg_data, posterior_channels)

    # FFT
    freqs, psd = signal.welch(posterior_data, sfreq, nperseg=2048)

    # Faixa alfa (8-13 Hz)
    alpha_range = (freqs >= 8) & (freqs <= 13)
    alpha_freqs = freqs[alpha_range]
    alpha_psd = psd[alpha_range]

    # Frequência dominante
    dominant_freq = alpha_freqs[np.argmax(alpha_psd)]

    # Amplitude média
    amplitude = np.mean(np.sqrt(alpha_psd))

    # Avaliar normalidade para idade
    expected_range = get_expected_range(age_years)
    is_normal = expected_range[0] <= dominant_freq <= expected_range[1]

    return {
        'frequency_hz': dominant_freq,
        'amplitude_uv': amplitude,
        'normal_for_age': is_normal,
        'expected_range': expected_range
    }

def get_expected_range(age_years):
    """
    Valores esperados por idade
    """
    if age_years < 3:
        return (5, 7) # Predominantemente theta
    elif age_years < 8:
        return (7, 9) # Transição
    else:
        return (8, 12) # Alfa maduro
```

Entregável Semana 20: ☒ Identificador de ritmo funcionando ☒ Calcula frequência corretamente (deve detectar 8.5 Hz no Isaac) ☒ Avalia normalidade para idade

 MÊS 5: GERAÇÃO DE LAUDOS

Semana 21-22: Template e Estrutura de Laudo

Responsável: Dev Backend + Médico (validação)

Objetivo: Criar estrutura padronizada de laudo

Template JSON:

```
json
```

```

{
  "laudo_template": {
    "identificacao": {
      "nome_paciente": "{{patient.name}}",
      "data_nascimento": "{{patient.birth_date}}",
      "idade": "{{patient.age_display}}",
      "indicacao": "{{exam.indication}}",
      "data_exame": "{{exam.date}}",
      "registro": "{{exam.id}}"
    },
    "informacoes_tecnicas": {
      "tipo_exame": "{{exam.type}}",
      "duracao_minutos": "{{exam.duration}}",
      "condicoes": "{{exam.conditions}}",
      "qualidade": "{{exam.quality}}"
    },
    "atividade_base": {
      "ritmo_posterior": "{{analysis.base_rhythm.description}}",
      "organizacao": "{{analysis.organization}}",
      "simetria": "{{analysis.symmetry}}",
      "reatividade": "{{analysis.reactivity}}"
    },
    "atividade_epileptiforme": {
      "presente": "{{analysis.epileptiform.present}}",
      "descricao": "{{analysis.epileptiform.description}}",
      "localizacao": "{{analysis.epileptiform.location}}",
      "frequencia": "{{analysis.epileptiform.frequency}}"
    },
    "conclusao": {
      "classificacao": "{{analysis.classification}}",
      "achados_principais": "{{analysis.main_findings}}"
    }
  }
}

```

Entregável Semana 22: ✅ Template estruturado e validado por médico ✅ Mapeamento de dados da análise para template

Semana 23-24: Integração com LLM (Claude API)

Responsável: Dev Python + Dev Backend

Objetivo: Gerar texto médico profissional

Implementação:

python

```
import anthropic

def generate_report_text(analysis_data, patient_data, exam_data):
    """
    Gera laudo em linguagem médica usando Claude
    """
    client = anthropic.Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

    # Montar prompt
    prompt = f"""
Você é um médico neurofisiologista experiente.
Escreva um laudo de eletroencefalograma seguindo o padrão brasileiro.

DADOS DO PACIENTE:
- Nome: {patient_data['name']}
- Idade: {patient_data['age']} anos
- Indicação: {exam_data['indication']}

DADOS TÉCNICOS:
- Duração: {exam_data['duration']} minutos
- Condições: {exam_data['conditions']}
- Qualidade: Satisfatória

ANÁLISE COMPUTADORIZADA:
- Classificação: {analysis_data['classification']}
- Ritmo de base: {analysis_data['base_rhythm']['frequency_hz']} Hz nas regiões poste
- Assimetria: {analysis_data['asymmetry']['description']}
- Paroxismos detectados: {len(analysis_data['spikes'])} eventos
  Localização: {analysis_data['spike_locations']}

INSTRUÇÕES:
1. Escreva o laudo completo seguindo a estrutura:
    - IDENTIFICAÇÃO
    - INFORMAÇÕES TÉCNICAS
    - ATIVIDADE DE BASE
    - ATIVIDADE EPILEPTIFORME
    - CONCLUSÃO

2. Use terminologia médica técnica apropriada
3. Seja objetivo e preciso
4. NÃO invente informações não fornecidas
5. Indique "EEG NORMAL" ou "EEG ANORMAL" na conclusão
    """

    message = client.messages.create(
        model="claude-sonnet-4-20250514",
```

```
        max_tokens=2000,  
        messages=[  
            {"role": "user", "content": prompt}  
        ]  
    )  
  
    laudo_text = message.content[0].text  
  
    return laudo_text
```

Custo Estimado:

- ~\$0.01-0.03 por laudo
- 100 laudos/mês = ~\$1-3/mês

Testes:

- ☐ Gerar laudo para caso Isaac
- ☐ Comparar com laudo real
- ☐ Validar com médico (qualidade aceitável?)
- ☐ Testar com 10 casos diferentes

Entregável Semana 24: ☒ Geração de laudo funcionando ☒ Qualidade aprovada por médico (70%+ aceitável) ☒ API integrada: POST /api/reports/generate

MÊS 6: INTERFACE FINAL E POLIMENTO

Semana 25-26: Tela de Revisão de Laudo

Responsável: Dev Frontend

Objetivo: Médico revisa e aprova laudo

Interface:

typescript

```

@Component({
  selector: 'app-report-review',
  template: `
    <div class="report-review">
      <!-- Lado esquerdo: EEG com marcações da IA -->
      <div class="eeg-panel">
        <app-eeg-viewer [examId]="examId"
                      [aiDetections]="aiDetections">
        </app-eeg-viewer>

        <h3>Achados da IA</h3>
        <mat-list>
          <mat-list-item *ngFor="let detection of aiDetections">
            <mat-icon [style.color]="detection.confirmed ? 'green' : 'orange'">
              {{ detection.confirmed ? 'check_circle' : 'help' }}
            </mat-icon>

            <div>
              <strong>{{ detection.time }}s - {{ detection.channel }}</strong>
              <p>{{ detection.type }} ({{ detection.confidence }}%)</p>
            </div>

            <button (click)="confirmDetection(detection)">✓</button>
            <button (click)="rejectDetection(detection)">✗</button>
          </mat-list-item>
        </mat-list>
      </div>

      <!-- Lado direito: Editor de laudo -->
      <div class="report-panel">
        <h2>Laudo Gerado</h2>

        <quill-editor [(ngModel)]="reportText"
                    [styles]="editorStyles">
        </quill-editor>

        <div class="actions">
          <button mat-raised-button (click)="saveAsDraft()">
            Salvar Rascunho
          </button>

          <button mat-raised-button
                color="primary"
                (click)="approveReport()">
            ✓ Aprovar e Finalizar
          </button>
        </div>
      </div>
    </div>
  `
})

```

```

        </div>
    </div>
</div>
,
})
export class ReportReviewComponent {
    examId: string;
    reportText: string;
    aiDetections: Detection[];

    confirmDetection(detection: Detection) {
        detection.confirmed = true;
        // Salvar feedback para retreino
        this.feedbackService.saveCorrection({
            detection_id: detection.id,
            correct: true
        });
    }

    rejectDetection(detection: Detection) {
        detection.confirmed = false;
        // Salvar como falso positivo
        this.feedbackService.saveCorrection({
            detection_id: detection.id,
            correct: false,
            reason: 'false_positive'
        });
    }

    approveReport() {
        // Finalizar laudo
        this.reportService.approveReport(this.examId, {
            final_text: this.reportText,
            detections: this.aiDetections,
            approved_by: this.currentUser.id,
            approved_at: new Date()
        }).subscribe(() => {
            this.router.navigate(['/exams']);
        });
    }
}

```

Entregável Semana 26: ☒ Interface de revisão completa ☒ Médico pode confirmar/rejeitar detecções ☒ Laudo editável ☒ Sistema salva feedback

Semana 27: Exportação de PDF

Responsável: Dev Backend

Objetivo: Gerar PDF profissional do laudo

Biblioteca: iTextSharp ou PdfSharpCore

csharp

```

public class PdfReportGenerator
{
    public byte[] GeneratePdf(Report report)
    {
        using var ms = new MemoryStream();
        var document = new Document(PageSize.A4);
        var writer = PdfWriter.GetInstance(document, ms);

        document.Open();

        // Cabeçalho
        var headerFont = FontFactory.GetFont("Arial", 16, Font.BOLD);
        document.Add(new Paragraph("LAUDO DE ELETROENCEFALOGRAMA", headerFont));
        document.Add(new Paragraph(" ")); // Espaço

        // Identificação
        AddSection(document, "IDENTIFICAÇÃO", new[] {
            $"Nome: {report.Patient.Name}",
            $"Data de Nascimento: {report.Patient.BirthDate:dd/MM/yyyy}",
            $"Idade: {report.Patient.Age} anos",
            $"Indicação: {report.Exam.Indication}",
            $"Data do Exame: {report.Exam.Date:dd/MM/yyyy}"
        });

        // Corpo do laudo
        document.Add(new Paragraph(report.FinalText));

        // Assinatura
        document.Add(new Paragraph(" "));
        document.Add(new Paragraph($"Laudado e revisado por:"));
        document.Add(new Paragraph($"Dr(a). {report.ApprovedBy.Name}"));
        document.Add(new Paragraph($"CRM: {report.ApprovedBy.CRM}"));
        document.Add(new Paragraph($"Data: {report.ApprovedAt:dd/MM/yyyy HH:mm}"));

        document.Close();

        return ms.ToArray();
    }
}

```

Entregável Semana 27: ☒ Geração de PDF funcionando ☒ Layout profissional ☒ Download de PDF

Semana 28: Testes Finais e Ajustes

Responsável: Equipe completa

Objetivo: Garantir qualidade antes do lançamento

Atividades:

Testes Funcionais:

☐ Testar fluxo completo end-to-end:

1. Upload de exame
2. Visualização
3. Análise com IA
4. Geração de laudo
5. Revisão médica
6. Aprovação
7. Download PDF

☐ Testar em diferentes navegadores (Chrome, Firefox, Safari, Edge)

☐ Testar PWA instalável

☐ Testar modo offline

Testes de Performance:

☐ Processar exame de 60 min em < 2 minutos

☐ Suportar 10 usuários simultâneos

☐ Upload de arquivo 50MB em < 30 segundos

☐ Renderizar gráfico em < 1 segundo

Testes de Segurança:

☐ Scan de vulnerabilidades (OWASP ZAP)

☐ Teste de injeção SQL

☐ Teste de XSS

☐ Validar HTTPS

☐ Validar autenticação

Testes com Médico:

☐ Médico testa com 10 casos reais

☐ Coletar feedback estruturado:

- Facilidade de uso (1-5)
- Qualidade dos laudos (1-5)
- Tempo economizado
- Sugestões de melhoria

Correções e Ajustes:

- ☐ Corrigir bugs encontrados
- ☐ Ajustar UX baseado em feedback
- ☐ Otimizar performance onde necessário

Documentação:

- ☐ Manual do usuário (PDF)
- ☐ Vídeo tutorial (5-10 min)
- ☐ FAQ
- ☐ Documentação técnica para desenvolvedores

Entregável Semana 28: ☒ Sistema testado e estável ☒ Bugs corrigidos ☒ Documentação completa ☒ Pronto para produção

 EQUIPE E RESPONSABILIDADES

 Composição da Equipe

Tech Lead / Arquiteto (1 pessoa)

Responsabilidades:

- Definir arquitetura técnica
- Code reviews
- Decisões técnicas críticas
- Mentoria da equipe
- Garantir qualidade do código

Perfil:

- 5+ anos de experiência
- Conhecimento em .NET, Angular, Python
- Experiência com arquitetura de sistemas
- Conhecimento em IA/ML (desejável)

Desenvolvedor Full-Stack .NET + Angular (1 pessoa)

Responsabilidades:

- Backend .NET 8 API
- Frontend Angular PWA
- Integração entre camadas
- Testes unitários e integração

Perfil:

- 3+ anos .NET Core/C#
 - 2+ anos Angular
 - Entity Framework Core
 - RESTful APIs
 - Git/CI-CD
-

Cientista de Dados / ML Engineer (1 pessoa)

Responsabilidades:

- Desenvolvimento modelos de IA
- Pré-processamento de sinais
- Treinamento e validação
- Pipeline de ML
- Python/FastAPI

Perfil:

- 2+ anos em Data Science
 - Python (NumPy, SciPy, scikit-learn)
 - ML/Deep Learning (TensorFlow ou PyTorch)
 - Processamento de sinais (desejável)
 - Conhecimento em MNE-Python (diferencial)
-

Médico Neurologista / Neurofisiologista (Consultor - 20h/mês)

Responsabilidades:

- Validação clínica
- Marcação de exames para treino

- Revisão de laudos gerados
- Feedback sobre interface
- Validação de detecções da IA

Dedicação:

- 5 horas/semana
 - 20 horas/mês
-

DevOps / Infraestrutura (Parcial - 10h/mês)

Responsabilidades:

- Setup de infraestrutura cloud
- CI/CD pipelines
- Monitoramento
- Backups
- Segurança

Pode ser:

- Consultor externo
 - Serviço gerenciado (AWS/Azure)
-

 Distribuição de Esforço

Mês 1-2: Fundação

- └ Tech Lead: 100h
- └ Dev Full-Stack: 160h
- └ Cientista Dados: 80h
- └ Médico: 40h
- └ DevOps: 20h

Mês 3-4: IA

- └ Tech Lead: 80h
- └ Dev Full-Stack: 120h
- └ Cientista Dados: 160h
- └ Médico: 40h
- └ DevOps: 10h

Mês 5-6: Interface e Polimento

- └ Tech Lead: 80h
- └ Dev Full-Stack: 160h
- └ Cientista Dados: 80h
- └ Médico: 40h
- └ DevOps: 20h

TOTAL 6 MESES:

- └ Tech Lead: 260h
- └ Dev Full-Stack: 440h
- └ Cientista Dados: 320h
- └ Médico: 120h
- └ DevOps: 50h

5 TECNOLOGIAS

◆ Backend

.NET 8 Web API

- **Versão:** .NET 8 LTS
- **Bibliotecas:**
 - Entity Framework Core 8.0
 - ASP.NET Core Identity (autenticação)
 - Swashbuckle (Swagger/OpenAPI)
 - FluentValidation (validações)
 - AutoMapper (mapeamentos)
 - Serilog (logging)
 - xUnit (testes)

🧩 Frontend

Angular 17

- **Framework:** Angular 17 (latest)
- **PWA:** @angular/pwa
- **UI:** Angular Material 17

- **Bibliotecas:**
 - RxJS (programação reativa)
 - NgRx (state management - opcional)
 - Chart.js ou Plotly.js (gráficos)
 - Quill Editor (editor de texto rico)
 - Jasmine/Karma (testes)
-

IA / Machine Learning

Python 3.11+

- **Framework API:** FastAPI 0.104+
 - **Processamento EEG:**
 - MNE-Python 1.6+ (leitura .EDF)
 - NumPy 1.25+
 - SciPy 1.11+ (filtros, FFT)
 - **Machine Learning:**
 - scikit-learn 1.3+ (ML clássico)
 - TensorFlow 2.14+ ou PyTorch 2.0+ (deep learning)
 - Pandas (manipulação dados)
 - **LLM:**
 - Anthropic SDK (Claude API)
 - **Testes:**
 - pytest
-

Banco de Dados

PostgreSQL 15+

- **ORM:** Entity Framework Core
 - **Migrações:** EF Core Migrations
 - **Backup:** Automated daily backups
-

Storage

Azure Blob Storage ou AWS S3

- Armazenamento de arquivos .EDF
 - PDFs de laudos
 - Modelos de IA treinados
-

DevOps

Containerização:

- **Docker** (desenvolvimento local)
- **Docker Compose** (orquestração local)

CI/CD:

- **GitHub Actions** ou **Azure DevOps**
- Pipeline: Build → Test → Deploy

Cloud (escolher):

- **Opção A: Azure**
 - App Service (.NET API)
 - Static Web Apps (Angular)
 - Container Instances (Python)
 - Database for PostgreSQL
- **Opção B: AWS**
 - Elastic Beanstalk (.NET API)
 - Amplify (Angular)
 - ECS/Fargate (Python)
 - RDS PostgreSQL

Monitoramento:

- Application Insights (Azure) ou CloudWatch (AWS)
 - Sentry (error tracking)
-

6 METODOLOGIA DE TRABALHO

5 Metodologia Ágil - Scrum Adaptado

Sprints

- **Duração:** 2 semanas
- **Total:** 12 sprints em 6 meses

Cerimônias

Planning (Segunda-feira início do sprint):

- Duração: 2 horas
- Definir objetivos do sprint
- Quebrar tarefas
- Estimar pontos (Planning Poker)

Daily Standup (Diário - 15 min):

- O que fiz ontem
- O que farei hoje
- Impedimentos

Review (Sexta-feira fim do sprint):

- Duração: 1 hora
- Demo do que foi feito
- Validação com médico (quando aplicável)

Retrospectiva (Sexta-feira fim do sprint):

- Duração: 1 hora
 - O que funcionou bem
 - O que melhorar
 - Ações para próximo sprint
-

Kanban Board (Jira ou GitHub Projects)

TODO | IN PROGRESS | CODE REVIEW | TESTING | DONE

Definition of Done

- ☐ Código escrito
 - ☐ Testes unitários (cobertura > 70%)
 - ☐ Code review aprovado
 - ☐ Documentação atualizada
 - ☐ Testado em ambiente de dev
 - ☐ Sem bugs críticos
-

Code Review

Processo:

1. Dev cria Pull Request
2. Tech Lead revisa código
3. Aprovação ou solicitação de mudanças
4. Merge após aprovação

Critérios:

- Clean code
 - Padrões do projeto
 - Performance
 - Segurança
 - Testes adequados
-

Documentação

Tipos:

- **README.md** em cada repositório
- **API Documentation** (Swagger auto-gerado)

- **User Manual** (para médicos)
- **Technical Docs** (arquitetura, decisões)

Ferramentas:

- Markdown
 - Confluence ou Notion
 - Swagger UI
-

7 ENTREGÁVEIS POR MÊS

Mês 1

- ☐ Projeto estruturado (Backend, Frontend, IA)
- ☐ Pipeline CI/CD básico
- ☐ Upload de arquivos .EDF funcionando
- ☐ Visualização de EEG (19 canais)
- ☐ Documentação inicial

Mês 2

- ☐ Pré-processamento de sinais completo
- ☐ Interface de marcação para médicos
- ☐ 20+ exames marcados para treino
- ☐ Database com estrutura completa

Mês 3

- ☐ Detector de paroxismos epileptiformes (acurácia > 75%)
- ☐ API de detecção funcionando
- ☐ Testes com casos reais

Mês 4

- ☐ Analisador de assimetrias
- ☐ Identificador de ritmo de base
- ☐ Pipeline completo de análise

Mês 5

- ☐ Geração de laudos com LLM

- ☐ Template estruturado
- ☐ Qualidade aprovada por médico

Mês 6

- ☐ Interface de revisão completa
 - ☐ Exportação de PDF
 - ☐ Sistema testado end-to-end
 - ☐ Documentação completa
 - ☐ **MVP PRONTO PARA USO**
-

RISCOS E MITIGAÇÕES

Riscos Técnicos

Risco 1: IA não atinge acurácia desejada (>80%)

Probabilidade: Média

Impacto: Alto

Mitigação:

- Começar com modelos simples (baseline)
 - Usar ML clássico antes de deep learning
 - Coletar mais dados se necessário
 - Ajustar threshold de confiança
 - Médico sempre revisa (segurança)
-

Risco 2: Performance inadequada (processamento lento)

Probabilidade: Baixa

Impacto: Médio

Mitigação:

- Otimizar código crítico desde início
- Usar cache agressivamente
- Processar em background
- Escalar recursos se necessário

Risco 3: Problemas de integração entre componentes

Probabilidade: Média

Impacto: Médio

Mitigação:

- Definir contratos de API cedo
 - Testes de integração contínuos
 - Comunicação frequente entre devs
-

Riscos de Projeto

Risco 4: Atraso no cronograma

Probabilidade: Média

Impacto: Alto

Mitigação:

- Buffer de 20% no planejamento
 - Priorizar features core (MVP mínimo)
 - Reuniões semanais de acompanhamento
 - Cortar features secundárias se necessário
-

Risco 5: Falta de dados para treino

Probabilidade: Baixa

Impacto: Alto

Mitigação:

- Acordar quantidade mínima com médico desde início
 - Usar datasets públicos como backup
 - Data augmentation se necessário
 - Transfer learning de modelos pré-treinados
-

Risco 6: Baixa adoção pelos médicos

Probabilidade: Baixa

Impacto: Alto

Mitigação:

- Co-criar com médico desde dia 1
 - Testes com usuários reais desde mês 4
 - Ajustar UX baseado em feedback
 - Treinamento adequado
-

Riscos de Negócio

Risco 7: Questões regulatórias (ANVISA)

Probabilidade: Média

Impacto: Alto

Mitigação:

- Posicionar como "auxílio diagnóstico"
 - Médico sempre tem palavra final
 - Documentar que IA sugere, médico decide
 - Consultor jurídico se necessário
-

Risco 8: Problemas de LGPD/privacidade

Probabilidade: Baixa

Impacto: Alto

Mitigação:

- Implementar segurança desde início
 - Anonimização de dados de treino
 - Consentimento explícito
 - Auditoria de acesso
 - Consultor LGPD se necessário
-

9 ORÇAMENTO (6 MESES)

👥 Equipe (Custo Principal)

Desenvolvedores:

Tech Lead / Arquiteto:

- Salário: R\$ 18.000/mês
- 6 meses: R\$ 108.000
- Encargos (40%): R\$ 43.200
- **Total:** R\$ 151.200

Dev Full-Stack (.NET + Angular):

- Salário: R\$ 15.000/mês
- 6 meses: R\$ 90.000
- Encargos (40%): R\$ 36.000
- **Total:** R\$ 126.000

Cientista de Dados / ML Engineer:

- Salário: R\$ 12.000/mês
- 6 meses: R\$ 72.000
- Encargos (40%): R\$ 28.800
- **Total:** R\$ 100.800

Médico Neurologista (Consultor):

- R\$ 250/hora
- 20 horas/mês × 6 meses = 120 horas
- **Total:** R\$ 30.000

DevOps (Parcial):

- R\$ 150/hora
- 10 horas/mês × 6 meses = 60 horas
- **Total:** R\$ 9.000

SUBTOTAL EQUIPE: R\$ 417.000

Infraestrutura e Ferramentas

Cloud (Azure ou AWS):

- Ambiente Dev: R\$ 300/mês
- Ambiente Staging: R\$ 500/mês
- Ambiente Prod: R\$ 1.000/mês (estimativa inicial)
- 6 meses: **R\$ 10.800**

Storage (Blob/S3):

- ~100 GB/mês (exames)
- R\$ 50/mês $\times$ 6 = **R\$ 300**

Database (PostgreSQL):

- Managed service: R\$ 400/mês
- 6 meses: **R\$ 2.400**

LLM API (Claude):

- ~500 laudos em testes
- R\$ 0.05/laudo = **R\$ 25**

Ferramentas de Desenvolvimento:

- GitHub (equipe): R\$ 200/mês $\times$ 6 = **R\$ 1.200**
- Jira/Confluence: R\$ 300/mês $\times$ 6 = **R\$ 1.800**
- Figma/Design: R\$ 100/mês $\times$ 6 = **R\$ 600**

SUBTOTAL INFRAESTRUTURA: R\$ 17.125

Datasets e Recursos

Datasets Públicos:

- Temple University Hospital EEG Corpus: **Grátis**
- CHB-MIT: **Grátis**

Compra de Datasets (backup):

- Caso necessário: R\$ 10.000

SUBTOTAL DATASETS: R\$ 10.000

🎓 Treinamento e Certificações

Cursos específicos:

- MNE-Python avançado: R\$ 500
 - TensorFlow especialização: R\$ 800
 - Total: R\$ 1.300
-

🔒 Segurança e Compliance

Consultoria LGPD:

- Revisão inicial: R\$ 5.000

Auditoria de Segurança:

- Pentest básico: R\$ 3.000

SUBTOTAL SEGURANÇA: R\$ 8.000

🚀 Contingência (15%)

- Buffer para imprevistos: R\$ 67.864
-

💰 ORÇAMENTO TOTAL

Categoria	Valor
Equipe	R\$ 417.000
Infraestrutura	R\$ 17.125

Categoria	Valor
Datasets	R\$ 10.000
Treinamento	R\$ 1.300
Segurança	R\$ 8.000
Contingência (15%)	R\$ 67.864
TOTAL 6 MESES	R\$ 521.289

Arredondado: R\$ 520.000

Distribuição Percentual

Equipe:	80%	(R\$ 417.000)
Infraestrutura:	3%	(R\$ 17.125)
Datasets:	2%	(R\$ 10.000)
Outros:	2%	(R\$ 9.300)
Contingência:	13%	(R\$ 67.864)

CRITÉRIOS DE SUCESSO

Critérios Funcionais (MVP)

1. Upload e Visualização:

- ☐ Sistema aceita arquivos .EDF
- ☐ Visualiza 19 canais simultaneamente
- ☐ Navegação fluida por exame de 60 minutos
- ☐ Performance: renderizar 10s em < 1 segundo

2. Análise com IA:

- ☐ Detecta paroxismos com sensibilidade > 80%
- ☐ Identifica assimetrias corretamente
- ☐ Calcula ritmo de base com erro < 0.5 Hz
- ☐ Classifica normal/anormal com acurácia > 75%

3. Geração de Laudos:

- ☐ Gera laudo estruturado automaticamente
- ☐ Linguagem médica profissional
- ☐ 70%+ do laudo aprovado sem edição
- ☐ Tempo de processamento < 2 minutos

4. Interface:

- ☐ Médico consegue revisar laudo em < 10 minutos
- ☐ Pode confirmar/rejeitar detecções da IA
- ☐ Editor de texto funcional
- ☐ Exporta PDF profissional

5. Funcionalidade Offline:

- ☐ PWA instalável
 - ☐ Visualiza exames em cache offline
 - ☐ Sincroniza quando volta online
-

☒ Critérios de Qualidade

Performance:

- ☐ Processar exame de 60 min em < 2 minutos
- ☐ Suportar 10 usuários simultâneos
- ☐ Uptime > 95%

Segurança:

- ☐ Autenticação JWT funcionando
- ☐ HTTPS obrigatório
- ☐ Dados criptografados
- ☐ Logs de auditoria

Usabilidade:

- ☐ Interface intuitiva (médico aprende em < 30 min)
 - ☐ Tempo de laudo reduzido em 50%+
 - ☐ Satisfação do usuário > 4/5
-

☒ Critérios de Validação Médica

Precisão Clínica:

- ☐ 0 erros críticos (não confundir normal com anormal grave)
- ☐ Falsos positivos aceitáveis (< 30%)
- ☐ Falsos negativos mínimos (< 20%)

Aceitação:

- ☐ 3-5 médicos testaram
 - ☐ 50+ exames processados
 - ☐ Feedback positivo (> 70% satisfação)
 - ☐ Laudos aprovados com mínima edição
-

☒ Critérios de Entrega

Documentação:

- ☐ Manual do usuário completo
- ☐ Vídeo tutorial gravado
- ☐ Documentação técnica
- ☐ API documentada (Swagger)

Testes:

- ☐ Cobertura de testes > 70%
 - ☐ Testes de integração passando
 - ☐ Testes end-to-end validados
 - ☐ Testado em produção com casos reais
-

PRÓXIMOS PASSOS

Semana 1: Kickoff

Reunião de Alinhamento (4 horas):

- Apresentar plano completo
- Alinhar expectativas
- Definir canais de comunicação
- Setup de ferramentas (Jira, Git, etc)

Responsáveis:

- Tech Lead

- Médico investidor
- Equipe de desenvolvimento

Entregável:

- Plano aprovado
 - Equipe alinhada
 - Ferramentas configuradas
-

Semana 2: Setup

Atividades:

- Criar repositórios Git
 - Setup ambientes de desenvolvimento
 - Primeira reunião de planning
 - Definir primeiro sprint
-

Acompanhamento

Reuniões Semanais:

- Sexta-feira, 14h-15h
- Review do sprint + planning

Comunicação:

- Slack para dia-a-dia
 - Jira para gestão de tarefas
 - GitHub para code review
-

CONCLUSÃO

Este plano de 6 meses é **ambicioso mas viável** com a equipe e recursos adequados.

Principais Pilares do Sucesso:

1.  **Equipe qualificada** com skills necessárias
2.  **Médico engajado** para validações

3. ☒ **Metodologia ágil** com entregas incrementais
4. ☒ **Foco no MVP** (sem features desnecessárias)
5. ☒ **Testes contínuos** com casos reais

Ao final dos 6 meses:

- Sistema funcional e testado
- IA com 75-85% de acurácia
- Redução de 50% no tempo de laudos
- Pronto para primeiros clientes

ROI Esperado:

- Investimento: R\$ 520.000
- Por cliente (R\$ 500/mês):
 - 100 clientes = R\$ 50.000/mês
 - Break-even em ~11 meses
 - Ano 2: R\$ 600.000/ano de receita

Preparado para começar! 🚀